SWE5006 · Team 8

Speaker Script and Live Demo Run Book

BlocBuy — Team 8 · Monday 26 October 2026, 16:30–17:15 SGT (Zoom)

Responsible	Yu Qianshuo (Yorik) (presentation lead, #464); slide master Lim Jiechao.
Companion files	`swe5006-presentation.html` — the deck (open in Chrome or Edge; **S** notes, **T** timer, **O** overview, **F** fullscreen, **B** backup index) · `swe5006-presentation.pdf` — the same deck as a PDF for submission.
Single source	`src/content.js` holds wording, timings, cues and Q&A; `src/slides-main.js` and `src/slides-backup.js` hold the slide layouts and `src/script.js` this document; shared UML diagrams come from `docs/presentation-kit/diagrams/`. Run `npm ci` once in `docs/presentation-kit/`, then `npm run all` in `docs/swe5006/presentation/`.
Status	Scaffold of 19 Oct 2026: placeholders until the SWE5006 guideline is applied. Slides due 23 Oct; rehearsal 25 Oct.

Markers

`[FILL]` value still to be supplied · `[VERIFY]` claim to check against the repo or report · `[CONFIRM]` needs team agreement. All must be gone after the 25 Oct rehearsal.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:30	15 s	1	BlocBuy	Lim	1a	13 s
16:30:15	40 s	2	Group buying only works if enough people join in time	Lim	1a	37 s
16:30:55	45 s	3	Scope: nine use cases on one platform	Lim	1b	44 s
16:31:40	50 s	4	Four sprints, one board, one Definition of Done	Lim	1e	49 s
16:32:30	50 s	5	Milestones and effort per member	Lim	1d	47 s
16:33:20	40 s	6	Status, open issues, and what each retro changed	Lim	1c	38 s
16:34	75 s	7	Top-down: split by transaction, one codebase, two topologies	Eric	2a	70 s
16:35:15	45 s	8	UC9 listings: State, Optimistic Offline Lock, and the schema	Liu Jin	2e	42 s
16:36	45 s	9	UC3/UC4 start and join: Factory Method and shared guards	Eric	2e	44 s
16:36:45	45 s	10	UC6 expiry refund: never refund twice	Yorik	2e	44 s
16:37:30	45 s	11	Notifications: Observer, Outbox, Idempotent Receiver	Pan	2e	44 s
16:38:15	45 s	12	UC8 My blocs: the URL owns the view state	Lim	2e	43 s
16:39	90 s	13	Seven kinds of tests, from domain rules to the deployed site	Liu Jin	3a	71 s
16:40:30	90 s	14	Gates that block a merge, and what tests caught	Liu Jin	3b	77 s
16:42	45 s	15	Source code and artefacts: one trunk, images by digest	Pan	4a	45 s
16:42:45	60 s	16	One pipeline from pull request to dev	Pan	4b	56 s
16:43:45	45 s	17	Security in the pipeline, no stored keys	Pan	4d	42 s
16:44:30	45 s	18	Where it runs: dev on EC2, prod on ECS	Pan	4b	42 s
16:45:15	45 s	19	Technical findings and issues	Pan	4c	41 s
16:46	15 s	20	Live: one change from pull request to dev	Pan	4b	14 s
16:53	20 s	21	Live demo: one bloc from invite to refund	Eric	5a	0 s
17:07	Q&A	22	Thank you	Lim	5b	

Segment talk budget vs slots vs estimated speaking time: §1 240 s / 240 s / 228 s · §2 300 s / 300 s / 287 s · §3 180 s / 180 s / 148 s · §4 660 s / 255 s / 240 s · §5 840 s / 20 s / 0 s (§5 talk is the demo itself).

1 Project Introduction & Project Management Summary (4+0 minutes)

Clock	16:30:00 – 16:34:00 (4 min talk + 0 min Q&A; talk ends 16:34:00)
Owner	Lim Jiechao
Slides	1, 2, 3, 4, 5, 6

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Explain the scope of the project.
1c — Explain the current status of the project including outstanding issues.
1d — Describe the actual project milestones and the total effort expended by each team member.
1e — Explain how many sprints planned, Definition of Done (DoD) achieved.
1f — Sprint retrospective & reflection in the next sprint
1g — Agile process artifacts like product backlog, user stories, sprint plans, burndown charts, sprint reviews and sprint retrospectives.

Slide 1 BlocBuy

16:30:00–16:30:15 · 15 s

Speaker Lim · covers guideline item **1a**

Say

Good afternoon, we're Team 8, and this is BlocBuy. I'll cover the problem and how we ran the project; then come design, testing, our pipeline, and a live demo.

≈ 29 words · ≈ 13 s at 130 wpm · slot 15 s

Cues

- 16:30:00 — Lim opens swe5006-presentation.html in Chrome, presses S for the notes window (keep it off the shared screen) and T there to start the timer. In Zoom, share only the deck window.
- Backchannel: the team chat stays open during the talk. Anyone who sees the share or audio drop posts there; Lim repeats any question asked during the talk in one line and names who answers (questions may come at any time, Q&A is 17:10–17:15).
- Tagline source: proposal §4 (two prices, pay first, automatic refund).

Hand-over *"So, what problem are we solving?"*

Slide 2 Group buying only works if enough people join in time

16:30:15–16:30:55 · 40 s

Speaker Lim · covers guideline item **1a**

Say

Small merchants can only offer a discount when volume is predictable, and a single buyer can never reach a bulk price. Advertising tries to bridge that gap, but it's expensive and not tied to real purchases. BlocBuy solves this with pay-first teams. Every listing has two prices. You pay upfront to start a team and share an invite link; friends join by paying too. If the team fills in time, everyone gets the team price. If not, everyone is refunded automatically.

≈ 81 words · ≈ 37 s at 130 wpm · slot 40 s · if short of time: If over time, drop "Advertising tries to bridge that gap… real purchases."

Cues

- Source: SWE5006 proposal §4 (27 Aug; the pay-first mechanic, two prices, the early Pinduoduo playbook) and §6.1 (Stripe test mode only). The README describes generic organisers and sellers; the proposal is the scope we built, so the slide follows it.
- What is unique: money is committed before the team exists, so every buyer has a reason to recruit, and the refund on expiry makes joining risk-free.

Hand-over *"Here's what we built to do that."*

Slide 3 Scope: nine use cases on one platform

16:30:55–16:31:40 · 45 s

Speaker Lim · covers guideline item **1b**

Say

This is our scope as a use case diagram. Buyers browse listings, sign in with Google, start or join a team, track their orders and teams, and receive notifications. Starting and joining both include paying upfront through Stripe Checkout. Joining through an invite link credits the person who shared it. A scheduler resolves each team, and an expired team triggers automatic refunds through Stripe. Admins manage listings and group terms. The initials show who owns each use case in our report. Out of scope: a merchant self-service GUI, analytics, recommendations, email or push, and live payments.

≈ 96 words · ≈ 44 s at 130 wpm · slot 45 s · if short of time: If over time, drop "The initials show who owns each use case in our report."

Cues

- Diagram: src/diagrams/use-case.svg. UC1–UC9 are the proposal's (§6.1); "Receive notifications" joined in Sprint 4 (#215, #446) and is Pan's report use case (#463). Out-of-scope list: proposal §6.2.
- «include»: START and JOIN share one Checkout path (CheckoutQuote accepts only START or JOIN, backend payment/model). «extend»: UC7 credits the sharer only when a join arrives through an invite link (proposal UC7); UC6 runs only when a team expires (proposal UC5, UC6).
- Initials = report owner per #463: LJ Liu Jin, WZ Eric, YQ Yorik, PS Pan, JC Lim. Beyond the proposal: the API-only admin (UC9) gained an admin console in Sprint 3 (PRs #305–#307, 30 Sep).

Hand-over *"And this is how we ran the project."*

Slide 4 Four sprints, one board, one Definition of Done

16:31:40–16:32:30 · 50 s

Speaker Lim · covers guideline item **1e**

Say

We planned four two-week sprints. Each sprint has a plan and a review, and from Sprint 3 we added a mid-sprint check; from Sprint 2, every working day has a dated log in the repository. The backlog is one GitHub Project board: 159 cards today, 132 of them done. We merged 356 pull requests, 70 of them Dependabot updates. We estimate in hours, 30 per member per sprint. Our Definition of Done grew after each retro: on dev means deployed, not local; pull requests stay under 1,500 lines; every quality gate blocks the merge. That strictness is why Sprint 3's goal does not count as met.

≈ 106 words · ≈ 49 s at 130 wpm · slot 50 s · if short of time: If over time, drop "We estimate in hours, 30 per member per sprint." (it is on slide 5).

Cues

- Counted 19 Oct with gh: board "GroupBuy Kanban Board" (project 2) 159 cards = 132 Done, 7 In progress, 15 Ready, 5 Backlog; 159 issues = 134 closed (119 completed, 15 not planned), 25 open; 356 merged PRs = 263 team, 70 Dependabot, 23 board-snapshot PRs. Sprint windows by merge date (SGT), Monday to the Sunday before the next sprint; Sprint 1 includes 7 PRs from 24 Aug.
- Sprint calendar: docs/agile/README.md (the proposal's dates differ by up to three days). Goals: Sprint 1 report, Sprint 2 plan (docx), docs/agile/sprint-03/plan.md §2, sprint-04/plan.md §2. Results: Sprint 2 report (9 of 11 stories, 134 of 150 h), sprint-03/review.md (117 of 150 h, goal not met on dev).
- Artefacts (item 1g): backlog = board + issue templates (use case, task, NFR); user stories with acceptance criteria in each plan; sprint plans in docs/agile/sprint-0N/plan.md (Sprints 1–2 as docx); burndowns from docs/agile/metrics/ (backup B11); reviews and retros in the progress reports. DoD text: sprint-03/plan.md §8, sprint-04/plan.md §8.
- Point at "Was it met?": the gates are required checks; the estimate field in the 12–19 Oct snapshots still holds each card's planned hours, not remaining hours.

Hand-over *"Here's how that adds up over time."*

Slide 5 Milestones and effort per member

16:32:30–16:33:20 · 50 s

Speaker Lim · covers guideline item **1d**

Say

Our milestones: the proposal on 27 August, the backend live on dev on 5 September, the web app on 22 September, and a release tag after each of the first three sprints. Sprint 2 finished 134 of 150 planned hours, Sprint 3 117 of 150. Sprint 4 carries 224.5 hours across two courses; by the 19 October snapshot, cards worth 118 planned hours were done, which is the burndown on the right. On the left are planned hours per member. We did not log actual hours per person, so this is the plan, against a guideline of about ten man-days each.

≈ 101 words · ≈ 47 s at 130 wpm · slot 50 s · if short of time: If over time, drop the milestone dates after 5 September.

Cues

- Sources: proposal (27 Aug); Sprint 1 report (dev live 5 Sep); Sprint 2 report (web app on dev 22 Sep; 150/134 h); tags v0.1.0-sprint1/2/3 (14 Sep, 27 Sep, 11 Oct; git tag); sprint-03/review.md (US34 listings on dev 8 Oct; 150 h, 33 h remaining at freeze); board: #455 and #456 Done in the 15 Oct snapshot.
- Burndown: docs/agile/metrics/2026-10-12…19.csv on the metrics/board-snapshots branch, the planned estimate of every Sprint 4 card not in Done (224.5 → 106.5 h); ideal line from sprint-04/plan.md §7. The 19 Oct file was captured early on 19 Oct, so it shows the state after the weekend.
- Per-member hours: Sprint 2 plan (30 h each), sprint-03/plan.md §4 (30 h each), sprint-04/plan.md §5 as planned (51, 48, 46, 41.5, 38).
  `[VERIFY: open PR #538 moves #468 and #458 after Planning: Liu Jin 66, Eric 26, Pan 46.5; use those once it merges]`
- `[CONFIRM: actual effort per member, agreed by all five; never present the planned hours as actuals]`
  Board items Done per assignee (19 Oct) count cards, not effort; a card shared by two people counts for both, and 21 Done cards have no assignee.

Hand-over *"So where does that leave us today?"*

Slide 6 Status, open issues, and what each retro changed

16:33:20–16:34:00 · 40 s

Speaker Lim · covers guideline item **1c**

Say

Where we stand on 19 October: the dev site is up, with the web app, sign-in and 14 listings. Stripe Checkout opens and returns, but the webhook never confirms the payment, so paid teams and refunds don't run on dev yet. That's issues 517 and 223, and our go/no-go for the live demo. Each retro changed the next sprint: no burndown in Sprint 1 led to daily board snapshots; Sprint 2's huge pull requests led to the 1,500-line rule and the mid-sprint check.

≈ 83 words · ≈ 38 s at 130 wpm · slot 40 s · if short of time: If over time, drop the second retro example (Sprint 2).

Cues

- Checked 19 Oct 14:14 SGT: curl https://d1b3np88z657cp.cloudfront.net/api/actuator/health → {"status":"UP"}; / → 200; /api/v1/products → 14 listings. Sign-in cookie: sprint-03/review.md (NFR-01 verified on dev, 8 Oct). #517 and #223 open on 19 Oct (gh issue view). Re-check all of this on 25 Oct and update the left column.
- Retro sources: Sprint 1 report (retrospective), Sprint 2 report §Sprint Retrospective → sprint-03/plan.md §8 (three new DoD rules). Sprint 3 retro is owed in #451; Sprint 4 retro is on 23 Oct.

Hand-over *"Eric will take you through the design."*

Q&A (during the talk / 17:10–17:15)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why does Sprint 3 count as not met when the code was merged?	Our Definition of Done says "on dev" means shown on the deployed environment, and a local run does not count. Checkout and scheduling were still off on dev at the freeze (#223), so the review recorded the goal as not met and carried it into Sprint 4 as the critical path. Evidence: sprint-03/review.md.	Lim
Are these hours actuals?	No. We planned in hours per story; done hours are planned hours of finished work (Sprint 2 report, Sprint 3 review), and the Sprint 4 burndown sums planned estimates of cards not yet Done. Actual hours per person were not logged, which is a limitation we state.	Lim
Was the effort balanced across the five members?	The plans gave everyone 30 h in Sprints 2 and 3; Sprint 4 started at 38–51 h each and was rebalanced after Planning (PR #538). Board cards differ in size, for example one Stripe pull request was about 10,000 lines, so card counts are not effort. `[CONFIRM: the effort split all five agree on]`	Lim
How did the retrospectives change what you did?	Each retro produced a rule in the next plan: daily board snapshots after Sprint 1; after Sprint 2, PRs under 1,500 lines, a mid-sprint check and remaining hours on the board. The last one was only partly kept: the estimate field still holds planned hours. Sources: Sprint 2 report, sprint-03/plan.md §8.	Lim
What is your biggest open risk for the demo?	The Stripe webhook on dev (#517): payments stay waiting, so paid START, JOIN and refunds do not complete on dev yet. Go/no-go for the live payment demo is 23 Oct, with a fallback recording by 25 Oct.	Lim → Yorik

2 Analysis & Design and Design Patterns (5+0 minutes)

Clock	16:34:00 – 16:39:00 (5 min talk + 0 min Q&A; talk ends 16:39:00)
Owner	Wang Ziang (Eric) with Liu Jin, Yu Qianshuo (Yorik), Panpakorn Siripanich (Pan), Lim Jiechao
Slides	7, 8, 9, 10, 11, 12

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

2a — Software architectural decisions that influence the logical architecture.
2b — Transition from analysis to design
2c — Use case diagram
2d — Analysis and design models
2e — Design problems and patterns
2f — Database schemas

Slide 7 Top-down: split by transaction, one codebase, two topologies

16:34:00–16:35:15 · 75 s

Speaker Eric · covers guideline item **2a**

Say *I'm Eric. We designed top-down, from the use cases on slide 3. First decision: a modular monolith. The code has five COLA modules, and each bounded context, identity, catalog, group buying, order, payment and notification, is a package in every module. Eighteen ArchUnit rules fail the build when a dependency points the wrong way. Second: split only where no transaction crosses. Paying, joining the team, creating the order and taking stock commit together, so those four contexts stay together as commerce. Third: one codebase, two topologies. For this course, dev runs the monolith on one EC2 host. For SWE5001, the same modules are assembled into three services on ECS, linked by an outbox and SNS and SQS. At the bottom is our rule from analysis to design: a context becomes a package, an entity becomes an aggregate with a repository port, and an external actor becomes a gateway port with an adapter.*

≈ 152 words · ≈ 70 s at 130 wpm · slot 75 s · if short of time: If over time, drop "Eighteen ArchUnit rules fail the build when a dependency points the wrong way." (it stays on the slide).

Cues

- Sources: backend/pom.xml (modules blocbuy-client, -adapter, -application, -domain, -infrastructure, -bootstrap; services/*-svc); backend/docs/design/backend-skeleton-plan.md (layer rules, gateway ports); ArchitectureTest.java (18 @ArchTest rules, e.g. domain_is_framework_free, commerce_reaches_identity_only_through_user_port, nothing_depends_on_notification); backend ADR 0002 (events, consequences before commit), 0007 (outbox, SNS, SQS), 0008 (two topologies, split follows transactional consistency).
- Point left (2a: modules and contexts), right (2a: topology), then the bottom strip (2b: transition rule). 2c: say "the nine use cases on slide 3"; do not go back to it. Worked transition examples per member: B7.
- Backups for questions: B1 context map, B2 aggregates, B9 logical view (ECS topology), B8 ADR index. ADR 0007 still reads "Proposed" in its file although it is merged and runs on ECS; say so if asked.

Hand-over *"Liu Jin takes the first use case."*

Slide 8 UC9 listings: State, Optimistic Offline Lock, and the schema

16:35:15–16:36:00 · 45 s

Speaker Liu Jin · covers guideline item **2e**

Say

I'm Liu Jin. Admins draft, publish and delist listings. Two problems: which moves are legal depends on the status, and two admins can edit one listing. For the first we use State: each ProductStatus constant overrides only the moves it allows; anything else is refused with a 409. For the second, Optimistic Offline Lock: every write sends the version the admin loaded, and the update matches only that version, so a stale edit gets a 412 and a Reload. On the right, our schema: 29 tables, grouped by owning context.

≈ 90 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "and two admins can edit one listing" from the problem sentence.

Cues

- `[VERIFY: Liu Jin to confirm against his §3.4/3.6 draft]` His draft is not in docs/swe5006/report-drafts/; the wording comes from the code.
- State: ProductStatus.java, enum constants override publish(), delist(), deletionError(), requireCommercialEdit(); the defaults throw CATALOG_409_002 ("State-pattern default"). Product.publish()/delist() call status.publish()/delist() and never branch on status. Tests: ProductStatusTest (onSalePublishThrowsStateConflict, offSaleDelistThrowsStateConflict).
- Lock: Product.requireVersion(expected) and BizProductMapper.xml "WHERE ... AND version = #{expectedVersion}"; ProductRepositoryImpl.requireChanged turns 0 rows into CATALOG_412_002 with {currentVersion}. expectedVersion is a body field (a query parameter on DELETE), not If-Match. Frontend ADR 0015: 412 → "You loaded v4; the server now has v5" banner and Reload. Test: AdminListingFlowIT.staleListingWritesReturnStoredVersionWithoutMutation.
- Schema (2f): 28 Flyway files V1–V28 in blocbuy-bootstrap/src/main/resources/db/migration, 29 CREATE TABLEs (V1 is a placeholder with the conventions). Grouping by owner follows backend ADR 0008. Full list: B5.

Hand-over *"Eric, starting and joining a team."*

Slide 9 UC3/UC4 start and join: Factory Method and shared guards

16:36:00–16:36:45 · 45 s

Speaker Eric · covers guideline item **2e**

Say

A team is created only after its first buyer has paid. Creation is a Factory Method: PaidGroupDealCreator's final initiate method checks the terms and the price, then calls createGroupDeal, which the standard creator implements. Our design problem was the join rules. They were written twice, for the quote and after payment, and they drifted: a team with no members passed the quote and failed after payment, so the buyer paid and was refunded. We moved the three rules into the GroupDeal aggregate; both paths call them. Honestly: there is one creator, wired at build time.

≈ 95 words · ≈ 44 s at 130 wpm · slot 45 s · if short of time: If over time, drop "Honestly: there is one creator, wired at build time." (it stays on the slide).

Cues

- `[VERIFY: PR #542 not merged; Eric confirms]` Draft: git show origin/docs/463-eric-wang-draft:docs/swe5006/report-drafts/eric-wang-contribution.md.
- Checked in code 19 Oct: PaidGroupDealCreator (abstract; public final initiate; protected abstract createGroupDeal), StandardPaidGroupDealCreator (GroupDeal.startPaid), GroupDealCreationConfig (@Bean, the only creator), PaidGroupDealService (final constructor-injected creator, #421); GroupDeal.requireJoinWindow, requireAvailableSeat, requireNewMember called by joinPaid and by GroupDealPurchaseSupportImpl.lockJoinTerms (#422). PR #424 merged 13 Oct.
- Tests: PaidGroupDealServiceTest.startUsesTheInjectedCreator; GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuoted (counts 0 and -1). Issue #463 words it "Factory Method / policy objects": policy objects were rejected because clients need the distinct codes DEAL_CLOSED, DEAL_FULL, ALREADY_MEMBER.

Hand-over *"Yorik takes it from expiry."*

Slide 10 UC6 expiry refund: never refund twice

16:36:45–16:37:30 · 45 s

Speaker Yorik · covers guideline item **2e**

Say *I'm Yorik. When a team expires, everyone is refunded. Expiry publishes TeamExpired, and two observers cancel the orders and record one refund per payment before the same commit. The risk: Stripe accepts a refund, our process dies before saving, and a naive retry refunds twice. So every call carries an idempotency key built from the obligation ID; a retry gets the same refund back. A worker claims each refund under a lease, retries after a fixed 30 seconds, and hands mismatches or anything past 23 hours to a human. Proven locally, not yet on dev.*

≈ 95 words · ≈ 44 s at 130 wpm · slot 45 s · if short of time: If over time, drop "and hands mismatches or anything past 23 hours to a human" (it stays on the slide).

Cues

- Source: docs/swe5006/report-drafts/yorick-yu-contribution.md, simplified. Code: RefundWork.providerKey() = "blocbuy-refund-" + id; StripeRefundGateway .setIdempotencyKey(work.providerKey()); RefundMapper.xml: claim sets lease_until +2 minutes and version+1, retry sets next_attempt_at +30 seconds, finish matches version; RefundService: ≥ 23 h since the first attempt → search Stripe only, no match → review; RefundWorkerJob every PT30S.
- Issue #463 says "Retry with Backoff"; the code is a fixed 30 s delay. Say "fixed delay", never "backoff". Observer: OrderLifecycleSubscriber @Order(10) and TeamExpiryRefundSubscriber @Order(20), both BEFORE_COMMIT (backend ADR 0002).
- Evidence limits: backend/docs/payment/stripe-refund-recovery-acceptance-20261003.md (local PostgreSQL, provider substitute); RefundServiceTest.lostResponseRetriesSameDurableKey. No refund on dev yet (#223, #517). Candidates and decisions: B6; sequence: B4.

Hand-over *"Pan, the notifications."*

Slide 11 Notifications: Observer, Outbox, Idempotent Receiver

16:37:30–16:38:15 · 45 s

Speaker Pan · covers guideline item **2e**

Say

I'm Pan. Buyers must hear what happened without commerce calling the notification service. The domain event is our Observer. The outbox writer stores an integration event in the same transaction, so it is never lost or announced after a rollback. A relay publishes it every two seconds: to SQS locally, or SNS then SQS on ECS. Delivery is at least once, so the consumer is an Idempotent Receiver: it inserts the event ID with ON CONFLICT DO NOTHING, and a duplicate writes nothing. Dev has no broker; this runs on ECS and in our tests.

≈ 95 words · ≈ 44 s at 130 wpm · slot 45 s · if short of time: If over time, drop ", so it is never lost or announced after a rollback".

Cues

- `[VERIFY: Pan confirms against his draft]` No draft in docs/swe5006/report-drafts/; wording from the code and backend ADR 0007.
- Code: IntegrationOutboxWriter (@TransactionalEventListener BEFORE_COMMIT, lowest precedence, only when blocbuy.events.enabled) → sys_outbox; OutboxRelayJob (fixedDelay 2 s, 2-minute lease) and OutboxRelayService (batch 100, 10 attempts); SysConsumedMessageMapper.xml "ON CONFLICT (event_id, consumer) DO NOTHING"; NotificationEventAppService.handle writes the marker and the notices in one transaction; DLQ after 5 receives (ADR 0007).
- Dev: application-ec2.yml defaults BLOCBUY_EVENTS_ENABLED, relay and consumer to false, and ADR 0007 records the scope decision of 11 Oct (notifications serve SWE5001 only), so dev produces no notices; do not demo them on dev. `[VERIFY: the dev host env file leaves BLOCBUY_EVENTS_* off]`
- Tests: OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT, ConsumedMessageRepositoryIT (firstMarkWinsAndRepeatsAreNoopsPerConsumer, concurrentFirstDeliveriesInsertExactlyOneMarker).

Hand-over *"Lim, the buyer's own page."*

Slide 12 UC8 My blocs: the URL owns the view state

16:38:15–16:39:00 · 45 s

Speaker Lim · covers guideline item **2e**

Say *Last, My blocs: the buyer's teams and orders on one page. A filter chip and a page number must survive a reload, the Back button and a shared link, and the API has no status filter. So the URL is the single source of view state. A schema validates the search parameters and falls back on bad values. The loader prefetches both lists, and switching a chip filters the cached list without refetching. One adapter function unwraps the response envelope at a single boundary. Liu Jin will now show how we tested all this.*

≈ 94 words · ≈ 43 s at 130 wpm · slot 45 s · if short of time: If over time, drop "One adapter function unwraps the response envelope at a single boundary." (it stays on the slide).

Cues

- `[CONFIRM: Lim picks the UC8 design problem]` Proposal from the code: URL as the single source of view state, plus one Adapter at the API boundary. The use-case section of docs/swe5006/report-drafts/lim-jiechao-contribution.md is still "to be written".
- Code: frontend/src/routes/_authenticated/account.tsx (validateSearch: accountSearchSchema, stripSearchParams defaults, loaderDeps page and size only, loader prefetches orders and blocs, redirect past the last page); features/blocs/blocs-search.ts (v.fallback to "active"); my-teams-query.ts (size 100, select unwrapResult); bloc-status.ts. Frontend ADRs 0001 (router owns URL state, Query owns server state), 0003 (generated client), 0009 (unwrapResult as select).
- Tests: frontend/src/routes/account.test.tsx: "switches chips through ?blocs= without refetching", "falls back to Active for an unknown ?blocs= value", "sends a page past the last one to the last page".
- Hand-over: segment 3 (testing) starts 16:39:00 with Liu Jin.

Hand-over *"Over to Liu Jin for testing."*

Q&A (during the talk / 17:10–17:15)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why a monolith on dev and services on prod?	Decision: one codebase, two topologies (backend ADR 0008). Reason: SWE5006 grades a complete runnable system, so dev stays the monolith on one EC2 host; SWE5001 needs three services on ECS. The services are thin assemblies of the same modules, and ArchUnit holds the service boundaries in the monolith too. Limitation: commerce still holds four contexts; extracting payment would need an outbox plus a saga.	Eric
Is ProductStatus really the State pattern, or an enum switch?	Decision: State, with enum constants as the concrete states. Reason: each constant overrides only the moves it allows and returns the next state; the defaults refuse with 409, and Product delegates without branching on status. Evidence: ProductStatus.java, ProductStatusTest. Contrast: GroupDeal uses a transition matrix (DealStatus.canTransitionTo), which we do not call State.	Liu Jin
How do you stop a double refund?	Decision: a durable obligation, then an idempotent call. The obligation is unique per payment and commits with the expiry; the Stripe idempotency key is built from its ID; a lease plus a version fence stops two workers; after 23 hours the worker only searches Stripe, never creates. Evidence:	Yorik

	RefundServiceTest.lostResponseRetriesSameDurableKey. Limitation: proven locally with a provider substitute, not on dev (#223).	
Why a fixed 30 s retry and not exponential backoff?	Decision: a fixed delay in RefundMapper.xml. Reason: the idempotency key makes each retry safe, and the 23-hour cutoff ends automatic retries. Limitation: there is no exponential backoff or jitter, so we call it a fixed delay, not "Retry with Backoff" as issue #463 words it.	Yorik
Why a Factory Method with only one creator?	Decision: keep the creator as a seam. Reason: initiate is final, so any creator keeps the validation, and constructor injection (#421) lets a test or configuration supply another creator without editing the service. Limitation: one creator, chosen at build time; we do not claim runtime-pluggable creation.	Eric
Where is the analysis model?	Slide 3 is the use-case model. Each member's report section has an analysis class and sequence diagram before the design ones (drafts in docs/swe5006/report-drafts/). Backup B7 maps analysis classes to design classes and ports per member; B1 is the context map and B2 the aggregates.	Eric
Is there a database per service?	On dev: one PostgreSQL database, schema "domain", all 29 tables. On ECS: one RDS instance with a database per service, and each runtime role is granted only its own tables (ADR 0008). Every database still applies the full Flyway stream while SWE5006 adds migrations, so foreign tables exist there but stay empty. Full list: B5.	Liu Jin
Do notifications work on dev?	No, by decision: the dev host runs no broker and leaves the outbox writer, relay and consumer off (application-ec2.yml; ADR 0007 scope decision, 11 Oct). The path runs on ECS through SNS and SQS, and locally and in CI against ElasticMQ (OutboxRelayElasticMqIT, NotificationConsumerElasticMqIT).	Pan

3 Types of Testing Conducted (3+0 minutes)

Clock	16:39:00 – 16:42:00 (3 min talk + 0 min Q&A; talk ends 16:42:00)
Owner	Liu Jin
Slides	13, 14

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

3a — Types of Testing Conducted
3b — Well structured source code with unit test cases for classes with more complex behaviours.

Slide 13 Seven kinds of tests, from domain rules to the deployed site

16:39:00–16:40:30 · 90 s

Speaker Liu Jin · covers guideline item **3a**

Say

Thanks, Lim. We test at seven levels; the pyramid shows how many of each. At the base, 182 unit test classes cover the domain and application layers, aimed at the classes with the most complex behaviour. RefundServiceTest proves that a lost Stripe response is retried with the same idempotency key, and PaidGroupDealTest checks 24 rules of a paid team. Above them, 49 integration test classes start real PostgreSQL 18, Redis and ElasticMQ in Testcontainers, so unique keys, version checks and the outbox run against the real database, not a mock. Eighteen ArchUnit rules hold the module boundaries, and the contract check regenerates the OpenAPI file from code and fails a breaking change. **[click]** *On the frontend, 76 Vitest files render components against MSW handlers generated from that same contract. Seven Playwright specs run on every pull request, and again against dev after each deploy. Performance is a scripted load test that we run by hand.*

≈ 154 words · ≈ 71 s at 130 wpm · slot 90 s · if short of time: If over time, drop "and PaidGroupDealTest checks 24 rules of a paid team".

Cues

- `[VERIFY: Liu Jin confirms the testing story]` No testing draft in docs/swe5006/repo drafts/; every number below was counted in the repository on 19 Oct at 8bcb38dd.
- Counts (19 Oct): find backend -name "*Test.java" = 183 (182 unit + ArchitectureT "*IT.java" = 49 (45 of them start Testcontainers: postgres:18, redis:8.10, softwaremill/elasticmq-native:1.7.1); grep -c @ArchTest ArchitectureTest.java = 1 frontend/src "*.test.ts*" = 76; frontend/e2e = 7 specs, 18 test() declarations. Stati @Test/@ParameterizedTest annotations: 1,281 in *Test, 241 in *IT. Per-module s backup B10.
- Examples: RefundServiceTest.lostResponseRetriesSameDurableKey (15 tests); PaidGroupDealTest (24); ProductStatusTest (5); OrderRepositoryIT.duplicatePaymentIsRejectedAtTheDatabase, saveIfVersionRejectsStaleWriteButAcceptsFreshOne; OutboxRelayElasticMqIT.publishesEventsWithAttributesAndRecoversAfterTranspor StripeRolloutAcceptanceIT (8; real HTTP, scheduler, Flyway and webhook signatur OAuth and Stripe network calls substituted).
- Where it runs: ci-backend.yml build-quality (mvnw verify: unit, ArchUnit, ITs) and contract-check (export, git diff --exit-code contracts/, openapi-diff 2.1.7 --fail- incompatible unless labelled contract-change-approved); ci-frontend.yml fronten check (pnpm check incl. Vitest), e2e-preview (PRs), e2e-dev (after deploy-s3-de Performance: backend/perf/workload.mjs, docs/perf/sprint3-baseline.md (local, 3 perf/k6 is the SWE5001 ECS load tool, only its tooling is tested in CI (perf-tooling.y Security scans (gitleaks, Trivy) are not counted here: Pan covers them.

Hand-over *"So what stops a bad change from merging?"*

Slide 14 Gates that block a merge, and what tests caught

16:40:30–16:42:00 · 90 s

Speaker Liu Jin · covers guideline item **3b**

Say

Tests only matter if they can stop a merge. On dev, six checks are required: the backend build with every test, the Docker image boot, gitleaks, the frontend check, and SonarCloud for both halves; the branch must be up to date first. SonarCloud blocks new code under 80 percent coverage or over 3 percent duplication. Today dev reads 99.2 percent on new backend code and 96.2 on frontend, and our latest full backend run passed 1,931 tests. **[click]** *Three things tests caught before production. Two red tests showed that a team with no members passed the quote, so a buyer could pay and then be refunded; that fix is on slide 9. ArchUnit failed the build when a new internal controller skipped input validation. And after a deploy, the dev smoke test got a 502: it ran while the single backend host restarted, so our dev deploys are not zero-downtime. Our limits are on the slide; Pan will now show the pipeline that runs all of this.*

≈ 166 words · ≈ 77 s at 130 wpm · slot 90 s · if short of time: If over time, drop the ArchUnit example (it stays on the slide).

Cues

- Required checks: gh api repos/PPsyrius/iss-groupbuy-team8/rulesets/22734681 (read 19 Oct): build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend; strict (up to date); 1 approval, threads resolved, squash only. contract-check, e2e-preview and Trivy run on PRs but are not required.
- Gate and coverage: docs/quality/quality-gates.md, NFR-06 (enforced). SonarCloud API, dev branch, read 19 Oct: backend gate OK, analysis 16 Oct (1f7f4d6a), coverage 99.4% (lines 99.6, branches 99.0), new code 99.2%; frontend gate OK, analysis 19 Oct (d9ff9e9d), coverage 96.1%, new code 96.2%; thresholds new_coverage < 80 and new_duplicated_lines_density > 3 fail. No repository-wide jacoco:check, by decision.
- Totals: docs/agile/sprint-04/log/2026-10-16.md (./mvnw -B verify, 1,931 tests, 0 failures, 1 opt-in skip, local with Docker on 24b4f47); 2026-10-19.md (pnpm check 799 tests; pnpm e2e 17 passed, 5 skipped). Cross-check: CI build-quality run 39132616205 (18 Oct, PR test/auth-surface) summed 1,962 tests, 1 skipped in 12:36 min.
- Caught: (1) sprint-03/log/2026-10-07.md "two zero/negative-count red tests reproduced the quote defect before the fix" (GroupDealPurchaseSupportImplTest.aTeamWithoutItsPaidLeaderCannotBeQuotec #422, merged in #424). (2) sprint-04/log/2026-10-13.md: controllers_are_validated required @Validated on InternalUserController (#468). (3) ci-frontend runs 37102432667 (3 Oct) and 37182996807 (4 Oct): e2e-dev smoke got 502 from /api/actuator/health at 06:17:51 and 06:33:03 UTC, inside deploy-existing-ec2 (06:17:23–06:18:03, 06:32:35–06:33:16) of the same commits.
- Limits: test-first for selected changes only (sprint-03/log/2026-10-03.md: rate-limit checks failed 429 vs 200 first; 2026-10-07.md); mutation checks only as hand probes (2026-10-12.md, 2026-10-13.md); load tests by hand; CodeQL job exists but repo variable ENABLE_CODEQL=false (security.yml).

Hand-over *"Pan, over to you for the pipeline demo."*

Q&A (during the talk / 17:10–17:15)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
How do you test payments without calling Stripe?	Decision: Stripe sits behind gateway ports (Checkout and Refund gateways). Reason: unit tests swap the port; StripeRolloutAcceptanceIT (8 tests) runs the real HTTP endpoints, scheduler, Flyway schema and webhook signature checks, with only OAuth and the Stripe network calls substituted, including the lost-response crash window (local rehearsal, 3 Oct). Limitation: no live Sandbox refund on dev yet (#223, #517).	Liu Jin → Yorik
Why Testcontainers rather than H2?	Decision: real PostgreSQL 18, Redis 8.10 and ElasticMQ in containers (45 of 49 IT classes). Reason: our invariants live in the database: UNIQUE keys, ON CONFLICT DO NOTHING, version-guarded UPDATEs, role grants; H2 would accept or reject different SQL. Evidence: OrderRepositoryIT.duplicatePaymentIsRejectedAtTheDatabase, ServiceGrantsIT (5/5 fail with the REVOKE disabled, 12 Oct log). Cost: Docker is needed; the CI build took 12:36 min on 18 Oct.	Liu Jin
Do you do TDD?	Not for everything, and we say so. Test-first for selected changes: the rate-limit checks failed (429 expected, 200 actual) before the code (3 Oct log); two red tests reproduced the join-quote defect before the fix (7 Oct log). Elsewhere tests come in the same pull request, and the 80% new-code gate makes them mandatory.	Liu Jin
What is your test coverage?	SonarCloud on dev, read 19 Oct: backend 99.4% overall and 99.2% on new code (analysis 16 Oct); frontend 96.1% overall and 96.2% on new code (19 Oct). The gate is at least 80% on new code (NFR-06); there is no repository-wide floor, by decision (quality-gates.md). Limitation: coverage is not assertion quality, and we have no automated mutation testing.	Liu Jin
How do you test the frontend against the API?	Decision: contract first. The backend regenerates contracts/openapi.json and fails on drift; openapi-diff fails a breaking change unless labelled. The frontend generates its client and its MSW handlers from that file (frontend ADRs 0003, 0011), and an unhandled request fails a test. Playwright mostly uses shared stubs; only smoke and the @auth/@admin cases reach the real backend. Limitation: stubs prove the frontend half only.	Liu Jin → Lim
Do you do performance testing?	Yes, by hand, not in CI. Local baseline 30 Sep (docs/perf/sprint3-baseline.md): 100 virtual users per scenario on real HTTP, PostgreSQL and Redis; p95 24 ms browse, 57 ms JOIN on one hot team, 18 ms My Orders, zero errors; 30 team expiries committed within 60 s. Local only, not a deployed result. k6 drives the SWE5001 ECS load test; the system demo shows it.	Eric

4 CI/CD Pipeline Demo (11+0 minutes)

Clock	16:42:00 – 16:53:00 (11 min talk + 0 min Q&A; talk ends 16:53:00)
Owner	Panpakorn Siripanich (Pan)
Slides	15, 16, 17, 18, 19, 20

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

4a — Describe the strategy to manage the source code and artifacts.
4b — Describe the DevSecOps pipeline and procedures.
4c — Highlight any technical findings and issues encountered.
4d — Design specification of the DevOps pipelines along with the use of appropriate tools with minimal security considerations.
4e — Backup: Record a video in advance in case of any technical issues during the live demo.

Slide 15 Source code and artefacts: one trunk, images by digest

16:42:00–16:42:45 · 45 s

Speaker Pan · covers guideline item **4a**

Say

Thanks, Liu Jin. One trunk: dev. Every change is a short feature branch and a pull request with a Conventional Commit title, squash-merged, so history stays linear. The ruleset needs one approval from a code owner: pipelines route to me, backend to Liu Jin, frontend to Lim. Threads must be resolved and six checks pass on an up-to-date branch. Dependabot proposes updates daily. Artefacts: the backend image sits in ECR with immutable tags and is deployed by digest, never by tag; the frontend is one static build with no per-environment settings; one OpenAPI file generates the client.

≈ 97 words · ≈ 45 s at 130 wpm · slot 45 s · if short of time: If over time, drop "Dependabot proposes updates daily." (it stays on the slide).

Cues

- `[VERIFY: Pan confirms the pipeline story]` Pan has no report draft; every fact on slides 15–20 comes from the repository, gh and AWS read-only calls of 19 Oct.
- Ruleset: gh api repos/PPsyrius/iss-groupbuy-team8/rulesets/22734681 (read 19 Oct) "dev-require-review": squash only, 1 approval, require_code_owner_review, dismiss_stale_reviews_on_push, required_review_thread_resolution, 6 strict required checks (build-quality, SonarCloud Backend, docker, gitleaks, frontend-check, SonarCloud Frontend), non_fast_forward, deletion; no bypass actors listed. CODEOWNERS: /.github/ @PPsyrius, /backend/ @l0sgAi, /frontend/ @limjiechao. Conventional Commits: CONTRIBUTING.md §Commits; PR template .github/PULL_REQUEST_TEMPLATE.md (evidence checklist).
- Dependabot: .github/dependabot.yml (maven, github-actions, docker, npm; daily; 7-day cooldown; Spring, Java, React and TypeScript majors ignored); 70 Dependabot PRs merged (slide 4). Tags: git tag = v0.1.0-sprint1/2/3; a v* tag runs the full ci-backend gate chain.
- Artefacts: ECR dev/blocbuy IMMUTABLE and scanOnPush (AWS CLI, 19 Oct); deployment/backend/ecr-lifecycle-policy.json keeps the newest 30; ci-backend.yml publish-ecr-dev pushes git-<sha>-run-<n> and checks the digest against ECR; deploy-ec2 action rejects a URI not pinned by sha256. Frontend: ADR 0004 (no VITE_*, CI grep), ci-frontend.yml deploy-s3-dev. Contract: contracts/openapi.json, contract-check; frontend ADR 0003 (generated client, not committed).
- SWE5001 prod branch: ruleset 24551375 "prod-release" (promotion PR only, merge commit, 3 required checks: promotion-source, service-images, infra-plan). Same project, other topology; say "SWE5001" if asked.

Hand-over *"Here is how a change travels."*

Slide 16 One pipeline from pull request to dev

16:42:45–16:43:45 · 60 s

Speaker Pan · covers guideline item **4b**

Say *On a pull request, the backend builds and runs every test, checks the API contract, then builds the image, boots it, checks its health endpoint and scans it with Trivy. SonarCloud blocks new code under 80 percent coverage. The frontend runs pnpm check, SonarCloud, and Playwright on its own build, and every pull request gets a secret scan. The blue boxes are the six required checks: about sixteen minutes for backend, five for frontend. After the squash merge, delivery rebuilds from the merged commit. The backend image goes to ECR by digest and reaches the host through SSM, with a health check and automatic rollback. The frontend syncs to S3 and is smoke-tested through CloudFront. Either is live in about three minutes.*

≈ 122 words · ≈ 56 s at 130 wpm · slot 60 s · if short of time: If over time, drop "SonarCloud blocks new code under 80 percent coverage." (Liu Jin covered it on slide 14).

Cues

- Diagram: src/diagrams/cicd-pipeline.svg, checked against .github/workflows/ci-backend.yml, ci-frontend.yml, security.yml and .github/actions/deploy-ec2/action.yml on 19 Oct.
- Timings, gh (19 Oct), medians of successful runs: backend PR start → docker end 964 s (18 runs, 16–19 Oct; build-quality 785 s, docker 170 s, contract-check 12 s); frontend PR start → e2e-preview end 289 s (19 runs; frontend-check 206 s, e2e-preview 69 s). Push to dev: backend start → deploy-existing-ec2 end 186 s (88 runs, 14 Sep–18 Oct; package-dev 46 s, publish-ecr-dev 73 s, deploy 40 s); frontend start → e2e-dev end 159 s (94 runs, 30 Sep–19 Oct; package 35 s, S3 21 s, e2e-dev 82 s). Example runs: backend deploy 39133506399 (18 Oct) 3 m 23 s = 55 s + 1 m 28 s + 43 s; frontend 39192166301 (19 Oct) merge → e2e-dev done 2 m 40 s, whole run 4 m 38 s because the post-merge Sonar scan runs alongside; backend PR 39132639003 16 m 29 s.
- Success, last 100 push runs: backend deploy job 88 of 97 (all 9 failures 14–17 Sep: SSM SendCommand InvalidInstanceId "Instances not in a valid state"; none since); frontend deploy-s3-dev 97 of 97, e2e-dev 94 of 97 (3–4 Oct, the 502s of slide 14). Full table: B12.
- Honest detail: the deployed jar/image and dist/ are rebuilt from the merged commit, not promoted from the PR run; Docker smoke and Trivy are not re-run after merge (docs/ci/backend-pipeline.md "Post-merge delivery boundary"). Strict up-to-date checks mean the merged tree is the tested tree; ECR scans the pushed image. Frontend ADR 0004 and the EC2 operator guide still describe promotion of the tested artefact: open item 18.
- Merge queue: merge_group triggers exist but are dormant (private personal repository, ci-frontend.yml comment). Deploy environments backend-dev and frontend-dev allow only dev and have no required reviewer, so the merge is the release decision.

Hand-over *"Security runs through every one of those boxes."*

Slide 17 Security in the pipeline, no stored keys

16:43:45–16:44:30 · 45 s

Speaker Pan · covers guideline item **4d**

Say

Security sits in each stage. Gitleaks is required on every pull request; SonarCloud and SpotBugs analyse the code; Trivy scans dependencies, configuration and the image. Every third-party action is pinned to a commit hash. There are no AWS keys in GitHub: three OIDC roles, trusted only from dev, each scoped to one job: push to one repository, run one command document on one instance, or sync one bucket. The container runs read-only with every capability dropped, and the account sits under organisation guardrails. Our gaps are on the bottom line.

≈ 90 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "Every third-party action is pinned to a commit hash." (it stays on the slide).

Cues

- Controls: security.yml (gitleaks on every PR and dev push; trivy-deps, trivy-config, workflow-lint, deployment-bundle path-filtered on PRs, all on v* tags and weekly Monday 02:17 UTC); ci-backend.yml build-quality (SpotBugs, Checkstyle, ArchUnit), docker (Trivy image HIGH/CRITICAL, ignore-unfixed, exit-code 1). 28 external actions, every one pinned to a 40-character SHA (grep of .github/workflows and .github/actions, 19 Oct).
- OIDC roles verified on AWS 19 Oct (orchestrator): BlocBuyGitHubEcrPush (trust: dev branch), BlocBuyGitHubBackendDeploy (trust: backend-dev environment; ssm:SendCommand on AWS-RunShellScript and one instance per docs/deploy/backend-ec2-pipeline-operator-guide.md §6 and deployment/backend/iam/github-ssm-deploy-policy.json.example), BlocBuyGitHubFrontendDeployDev (trust: frontend-dev environment; one bucket and one distribution). Both environments allow only the dev branch (gh api, 19 Oct).
- Runtime: deployment/backend/compose.yml read_only, cap_drop ALL, no-new-privileges, /tmp tmpfs, host network; secrets only in /etc/blocbuy/backend.env root:root 0600 (operator guide §7). Account: docs/deploy/aws-account-governance/README.md (Org-Baseline and Region-Pin on Root, Sandbox-Guardrails on the Sandbox OU; Identity Center, 1-hour sessions).
- Gaps: ENABLE_CODEQL=false (gh variable list, 19 Oct) and the code-scanning API answers "Code scanning is not enabled for this repository"; NFR-37 / #227 open (docs/deploy/dev-cloudfront-deployment.md §13); dev EBS unencrypted (AWS, 19 Oct); Gitleaks scans event-selected commits and a 14 Oct full-history audit left 24 old matches undecided (docs/quality/quality-gates.md). Two IAM users for SSM port forwarding, one active key (governance README): CI has no keys, the account does. Full list: B14.

Hand-over *"And this is where it all lands."*

Slide 18 Where it runs: dev on EC2, prod on ECS

16:44:30–16:45:15 · 45 s

Speaker Pan · covers guideline item **4b**

Say

This is where dev runs, all in Singapore. The browser only talks to CloudFront. Static files come from a private S3 bucket through origin access control. API calls go to a load balancer that accepts only CloudFront's address range and answers 403 without our origin-secret header. Behind it, one EC2 host runs the backend container from Docker Compose, next to PostgreSQL and Redis on localhost. Logs go to CloudWatch for fourteen days. On the right is SWE5001's production: the same code as three services on ECS Fargate, rebuilt on every release.

≈ 91 words · ≈ 42 s at 130 wpm · slot 45 s · if short of time: If over time, drop "for fourteen days".

Cues

- Diagram: src/diagrams/dev-deployment.svg. AWS facts read-only on 19 Oct (orchestrator; aws --profile blocbuy --region ap-southeast-1 describe/get/list): CloudFront E3U5W7UWDL8LRV d1b3np88z657cp.cloudfront.net, HTTP/2, no WAF, origins S3 blocbuy-dev-frontend-402178232802 (/*) and ALB blocbuy-dev-backend (/api/*); ALB internet-facing, 2 AZs, HTTP:80, default fixed 403, one forward rule on an http-header condition; SG ingress 80 only from com.amazonaws.global.cloudfront.origin-facing; target group HTTP 8589, health /api/actuator/health; EC2 i-0852b6c3475b19f05 "BlocBuy-dev" t3.large x86_64, ap-southeast-1b, Ubuntu 24.04, SSM online, 40 GB gp3; ECR dev/blocbuy; log group /blocbuy/backend/dev, 14 days; no CloudWatch alarm for dev (all six are SWE5001's).
  `[VERIFY: re-run the AWS read-only checks on 25 Oct (this draft could not: the CLI session had expired)]`
- Repo sources: docs/deploy/dev-cloudfront-deployment.md (OAC blocbuy-dev-frontend-oac, X-Origin-Secret rule, security headers policy, NFR-37); docs/deploy/backend-ec2-pipeline-operator-guide.md §1 (PostgreSQL and Redis on 127.0.0.1, pre-existing, not pipeline-managed; the Compose file holds only the backend, enforced by security.yml deployment-bundle); docs/deploy/backend-log-observability.md (PostgreSQL 18, Redis 8 on the host).
- Not shown: a second CloudFront dcsjrkmummxi7.cloudfront.net → S3 blocbuy-media-dev-402178232802 with a WAF web ACL exists in the account, but no document or code in the repository names it (grep, 19 Oct): open item 17. Stripe and Google are external callees of the backend (B13 shows them).
- Prod box: SWE5001 topology (shared physical-deployment diagram, backup B13; backend ADR 0008).

Hand-over *"Here is what building this taught us."*

Slide 19 Technical findings and issues

16:45:15–16:46:00 · 45 s

Speaker Pan · covers guideline item **4c**

Say *Four findings. First, a green pipeline is not a working feature: on dev, Stripe Checkout opens and returns, but the webhook never confirms the payment. That's issue 517, still open. Second, every backend deploy restarts the only host, so a smoke test once got a 502; we accept that on dev. Third, the organisation guardrails also block us as operators, so IAM changes go through the CLI. Fourth, from SWE5001: tests with mocks passed, but the real cloud failed, on a missing permission and a read-only temp folder.*

≈ 88 words · ≈ 41 s at 130 wpm · slot 45 s · if short of time: If over time, drop the third finding (guardrails) from speech; it stays on the slide.

Cues

- (1) #517 open (16 Oct: payment 01a14345-… stays waiting; likely causes in the issue: Sandbox endpoint URL, signing secret or subscribed events); #223 open (Checkout, scheduling and expiry refunds on dev). Owner Yorik; go/no-go 23 Oct (open item 1).
- (2) ci-frontend runs 37102432667 (3 Oct) and 37182996807 (4 Oct): e2e-dev got 502 from /api/actuator/health inside deploy-existing-ec2 of the same commit (slide 14). Decision: accept for dev (single host, container replaced in place, compose.yml stop_grace_period 30 s); not re-tested since.
- (3) docs/agile/sprint-02/log/2026-09-23.md "SCPs block bucket-list and Access Analyzer views but not scoped deployment"; docs/deploy/backend-log-observability.md troubleshooting (Access Analyzer denied → apply IAM with aws iam create-policy-version); consolidated to three SCPs on 16 Oct (#530, governance README).
- (4) SWE5001: PR #527 (16 Oct): Terraform tests on mocks passed, the real Up run 38888232111 failed because the apply role lacked application-autoscaling:TagResource; docs/agile/sprint-03/log/2026-10-06.md: read-only root on Fargate left /tmp owned by root, Tomcat could not start; fix VOLUME ["/tmp"] in both Dockerfiles. The same image worked on the dev host because Compose mounts a 1777 tmpfs.
- Also true, in B12: the first nine dev deploys (14–17 Sep) failed when SSM refused the instance ("not in a valid state"); none of the 88 since failed.

Hand-over *"Now let me show you one change going through."*

Slide 20 Live: one change from pull request to dev

16:46:00–16:46:15 · 15 s

Speaker Pan · covers guideline item **4b**

Say

Now live: one small frontend change. Checks take about five minutes; after approval and a squash merge, it's on dev in under three. The backend run and a fallback are recorded.

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

Cues

- Opens the live pipeline demo (about 405 s, run book in the script). `[CONFIRM: the demo change, its author and the approving code owner (frontend → Lim)]`
  Open the PR before 16:42 so its checks finish during slides 15–19.
- Numbers: frontend PR checks median 289 s (19 runs, 16–19 Oct); merge → e2e-dev green median 159 s (94 runs); backend merge → deployed median 186 s (88 runs). Recorded backend run: 39133506399 (18 Oct).
- Item 4e: `[FILL: fallback recording link and date (due 25 Oct, open item 2)]`

Hand-over (demo) "Here is the pull request."

Q&A (during the talk / 17:10–17:15)

Likely question	Answer: decision → reason → evidence or limitation (30–45 s)	Owner
Why EC2 and not ECS for dev?	Decision: dev reuses one existing EC2 host with the monolith in Docker Compose (operator guide; backend ADR 0008). Reason: SWE5006 needs one complete, always-on system at low cost, and the monolith fits one t3.large; ECS is the SWE5001 topology, rebuilt per release. Limitation: one host, so a deploy restarts the API (502 on 3–4 Oct), and PostgreSQL and Redis share the host outside the pipeline.	Pan
How do you roll back?	Automatic: deploy.sh health-checks the new container and restarts the previous digest if it fails; the workflow also checks public health and rolls back. Manual: rollback-backend.yml (reason, confirmation, same non-cancellable concurrency group). Limitation: rollback-backend.yml has never run (gh, 19 Oct) and no automatic rollback has fired (every red deploy failed earlier, at SSM). Flyway is never reversed, so migrations stay backward-compatible. Frontend: redeploy the previous commit (sync uses --delete).	Pan
How are secrets handled?	CI holds no AWS keys: OIDC gives short-lived roles per job. The repository has three Actions secrets, none for AWS or the app: SONAR_TOKEN, TYPESAFE_API_KEY and BOARD_SNAPSHOT_TOKEN (gh secret list, 19 Oct). Dev application secrets live in /etc/blocbuy/backend.env (root, 0600) on the host, never in GitHub or an SSM command; SWE5001 prod uses Parameter Store. Gitleaks gates every PR. Limitations: the host file is maintained by hand, and the account still has two IAM users for SSM port forwarding, one with an active key (governance README).	Pan
Why is CodeQL off?	The job exists in security.yml behind ENABLE_CODEQL, which is false: code scanning is not enabled for this private repository (GitHub API, 19 Oct), and the operator guide says to require CodeQL only once it is. SAST is covered by SonarCloud on every PR plus SpotBugs in the build. We list it as a gap (B14).	Pan
How long from merge to live?	Medians from gh on 19 Oct: backend about 3.1 minutes from the push run starting to the deploy job ending (88 runs); frontend about 2.7 minutes including the e2e-dev smoke (94 runs). Before that, PR checks take about 16 minutes for backend and 5 for frontend. Per-workflow table: B12.	Pan
What stops a bad deploy?	Before merge: six required checks on an up-to-date branch and a code-owner approval. After merge: the image URI must be an ECR digest in the expected repository, the host health check with automatic rollback, the public health check,	Pan

	and the e2e-dev smoke through CloudFront. ENABLE_EC2_DEPLOY and ENABLE_S3_DEPLOY freeze delivery. Limitation: the delivered image is rebuilt after merge and not rescanned by Trivy (ECR scans on push), and the deploy environments have no required reviewer.	
How do you manage dependencies?	Dependabot checks Maven, npm, Docker and GitHub Actions daily with a 7-day cooldown and groups (Spring, test, TanStack, HeroUI); majors of Spring, Java, React and TypeScript are ignored on purpose. Every update passes the same required checks; 70 Dependabot PRs merged. Trivy scans dependencies on change and weekly. Actions are pinned to commit SHAs, so Dependabot bumps the SHA.	Pan → Lim

5 System Demo (14+5 minutes)

Clock	16:53:00 – 17:12:00 (14 min talk + 5 min Q&A; talk ends 17:07:00)
Owner	Wang Ziang (Eric) and Yu Qianshuo (Yorik) (drive)
Slides	21, 22

Lecturer's presentation sequence · Presentation Guidelines · Briefing deliverables (V1.0)

5a — Demonstrate some of the significant use cases of the solution.
5b — Demonstrate how the solution deals with some of the important quality attributes e.g. scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).
5c — Backup: Record a video in advance in case of any technical issues during the live demo.

Slide 21 Live demo: one bloc from invite to refund

16:53:00–16:53:20 · 20 s

Speaker Eric · covers guideline item **5a**

Say `[FILL: draft wording]`

≈ 1 words · ≈ 0 s at 130 wpm · slot 20 s

Slide 22 Thank you

17:07 · Q&A

Speaker Lim · covers guideline item **5b**

Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Demo plan (draft)

Live on the deployed `dev` environment, with a fallback recording made by 25 Oct.

Step	Shows	Fallback
1	Invite a buyer to a team	`[FILL: recording]`
2	START or JOIN with Stripe Sandbox checkout	`[FILL: recording]`
3	`/account`: orders and blocs	`[FILL: recording]`
4	Expiry refund	`[FILL: recording]`

Open items

There are 6 `[FILL]`, 8 `[VERIFY]` and 4 `[CONFIRM]` markers in this script.

#	Item	Owner	Blocks slide
1	Enable Checkout, scheduling and expiry refunds on dev (#223) and fix the Stripe webhook (#517); go/no-go for the live payment demo on 23 Oct	Yorik	21
2	Fallback recordings of both demos on the deployed dev environment by 25 Oct	Yorik (system), Pan (pipeline)	20, 21
3	Sprint 1 planned and done hours: none were recorded; reconstruct from #49 or state "not tracked"	Pan	5
4	Sprint 3 retrospective (#451): fill the Sprint 3 → 4 row	Lim	6
5	Sprint 4 result, done hours, retrospective and the v0.1.0-sprint4 tag date after the 23 Oct review	Eric	4, 5, 6
6	Sprint 4 per-member planned hours once PR #538 merges (Liu Jin 66, Eric 26, Pan 46.5)	Eric	5
7	Actual effort per member, agreed by all five	Lim	5, Q&A 1
8	Re-check slide 6 status on 25 Oct (dev health, #517, #223, listings)	Lim	6
9	Confirm slide 8 (UC9 State, Optimistic Offline Lock, schema) and B5 against the §3.4–3.6 draft	Liu Jin	8, B5
10	Confirm slide 9 against the UC3/UC4 draft once PR #542 merges	Eric	9
11	Confirm slide 11 against the notifications draft; check the dev host env file leaves BLOCBUY_EVENTS_* off	Pan	11
12	Pick the UC8 design problem (proposal: URL as the single source of view state, one Adapter at the API boundary)	Lim	12
13	Confirm the testing story on slides 13–14 and B10 (no testing draft yet); recount the test numbers on 25 Oct	Liu Jin	13, 14, B10
14	Confirm the pipeline story on slides 15–20 and B12–B14 (no DevSecOps draft yet)	Pan	15–20, B12–B14
15	Choose the live demo change, its author and the approving code owner; open the PR before 16:42	Pan, Lim	20
16	Re-run the AWS read-only checks behind slide 18 and B14 on 25 Oct (this draft could not: CLI session expired)	Pan	18, B14
17	Document or remove the media CloudFront dcsjrkmummxi7 and bucket blocbuy-media-dev-402178232802 (nothing in the repository names them)	Pan	18
18	Reconcile frontend ADR 0004 and the EC2 operator guide with the post-merge rebuild in ci-frontend.yml and ci-backend.yml; CONTRIBUTING.md still lists contract-check as required	Pan, Lim	15, 16, B14
19	Fallback recording link for the pipeline demo (item 4e), part of open item 2	Pan	20